编译器设计专题实验三：LR(0) 项目集规范族构造

计算机2203 苏佳琪 2226124199

一、实验目的

本实验实现给定上下文无关文法的 LR(0) 项目集规范族构造。LR(0) 项目集规范族是构造 SLR(1) 分析表的基础，通过 Closure 和 Goto 操作可以得到语法分析器的状态集合以及状态之间的转移关系。

二、实验内容

本实验完成以下内容：

1. 从文件读取文法规则；
2. 支持产生式右部使用 `|` 表示多个候选式；
3. 自动增广文法，添加新的开始产生式；
4. 构造初始项目集 `I0`；
5. 实现 Closure 闭包计算；
6. 实现 Goto 状态转移计算；
7. 输出所有 LR(0) 项目集及其转移关系；
8. 为实验四的 SLR(1) 分析表生成提供基础。

三、输入文法

实验使用经典表达式文法作为测试：

```
E -> E + T | T
T -> T * F | F
F -> ( E ) | id
```

程序会自动增广为：

```
E' -> E
E -> E + T | T
T -> T * F | F
F -> ( E ) | id
```

四、核心算法

1. Closure 操作

如果项目集中存在项目：

```
A -> α · B β
```

其中 `B` 是非终结符，则需要将所有以 `B` 为左部的产生式加入项目集：

```
B -> · γ
```

重复该过程，直到项目集不再变化。

2. Goto 操作

对于项目集 `I` 和文法符号 `X`，如果存在项目：

```
A -> α · X β
```

则移动圆点得到：

```
A -> α X · β
```

然后对移动后的项目集求 Closure，得到新的项目集。

3. 规范族生成

从 `I0 = Closure({E' -> · E})` 开始，对每个项目集和每个文法符号计算 Goto。如果得到的新项目集不存在，则加入规范族。重复该过程，直到不再产生新的项目集。

五、运行方式

```
make
./compiler lr0 input/grammar_expr.txt output/lr0_items.txt
```

输出文件为：

```
output/lr0_items.txt
```

```
PS E:\course\4.2\CompilerProj\compiler_project> .\compiler.exe lr0 input\grammar_expr.txt output\exp3_lr0_items
written to output\exp3_lr0_items.txt
PS E:\course\4.2\CompilerProj\compiler_project> Get-Content output\exp3_lr0_items.txt
I0:
  E' -> 路 E
  E -> 路 E + T
  E -> 路 T
  T -> 路 T * F
  T -> 路 F
  F -> 路 ( E )
  F -> 路 id
  Goto:
    ( -> I1
    E -> I3
    F -> I4
    T -> I5
    id -> I2
----------------
I1:
  E -> 路 E + T
  E -> 路 T
  T -> 路 T * F
  T -> 路 F
  F -> 路 ( E )
  F -> ( 路 E )
  F -> 路 id
  Goto:
    ( -> I1
    E -> I6
    F -> I4
    T -> I5
    id -> I2
----------------
I2:
  F -> id 路
  Goto:
----------------
I3:
  E' -> E 路
  E -> E 路 + T
  Goto:
    + -> I7
----------------
I4:
  T -> F 路
  Goto:
----------------
I5:
  E -> T 路
  T -> T 路 * F
  Goto:
    * -> I8
----------------
I6:
```

```
Problems   Output   Debug Console   Terminal   Ports

  Goto:
    * -> I8
----------------
I6:
  E -> E 路 + T
  F -> ( E 路 )
  Goto:
    ) -> I9
    + -> I7
----------------
I7:
  E -> E + 路 T
  T -> 路 T * F
  T -> 路 F
  F -> 路 ( E )
  F -> 路 id
  Goto:
    ( -> I1
    F -> I4
    T -> I10
    id -> I2
----------------
I8:
  T -> T * 路 F
  F -> 路 ( E )
  F -> 路 id
  Goto:
    ( -> I1
    F -> I11
    id -> I2
----------------
I9:
  F -> ( E ) 路
  Goto:
----------------
I10:
  E -> E + T 路
  T -> T 路 * F
  Goto:
    * -> I8
----------------
I11:
  T -> T * F 路
  Goto:
----------------
PS E:\course\4.2\CompilerProj\compiler_project>
```

六、实验结果

输出结果包含所有项目集和转移关系，格式如下：

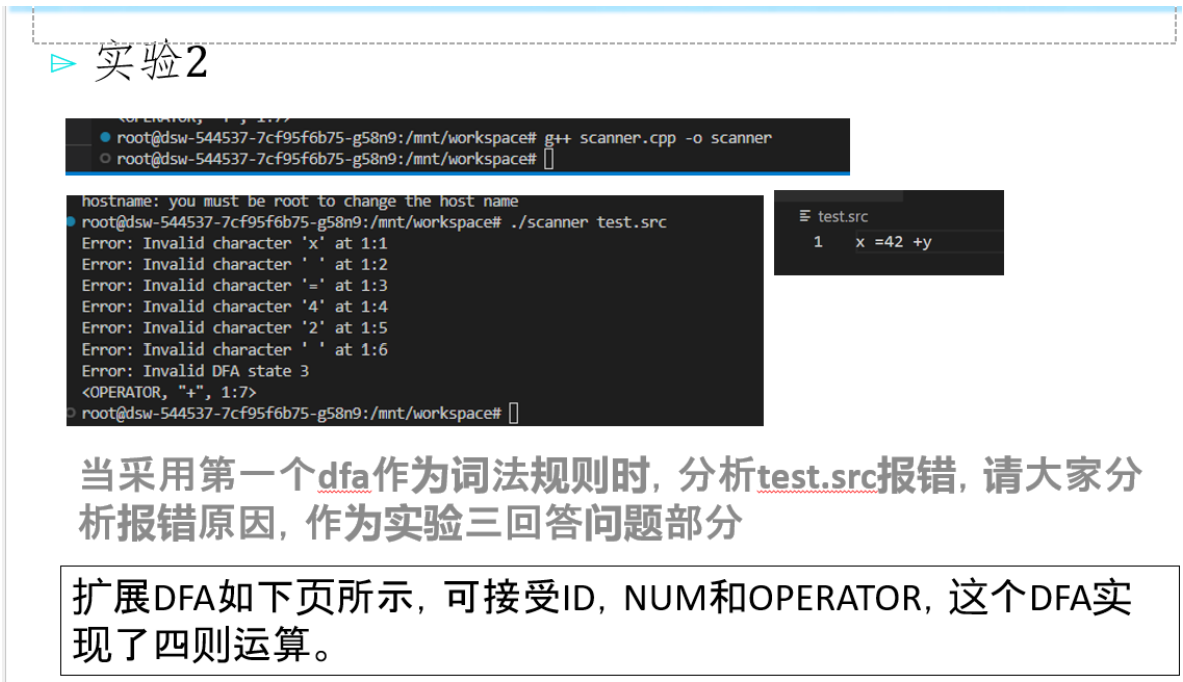
```
I0:
  E' -> · E
  E -> · E + T
  E -> · T
  T -> · T * F
  T -> · F
  F -> · ( E )
  F -> · id
  Goto:
    ( -> I1
    E -> I2
    F -> I3
    T -> I4
    id -> I5
----------------
```

每个 `Ii` 表示一个 LR(0) 项目集，`Goto` 部分表示该状态读入某个文法符号后的状态转移。

七、实验总结

本实验实现了 LR(0) 项目集规范族的自动构造。通过 Closure 和 Goto 两个核心操作，可以把文法转换为语法分析器中的状态机结构。该实验为下一步构造 SLR(1) ACTION/GOTO 分析表奠定了基础。

八、回答问题



采用第一个 DFA 作为词法规则时，分析 `test.src` 为什么会报错?

当实验二直接采用实验一中的第一个 DFA 作为词法分析规则时，分析 `test.src` 会报错，主要原因是实验一中的 DFA 只是用于识别特定规则字符串的简单自动机，并不是完整的词法分析 DFA。实验一 DFA 的字母表通常只包含少量字符，例如 `{a, b}`，它只能判断由这些字符组成的字符串是否属于该 DFA 定义的语言。但是 `test.src` 中的源程序类似于 `x = 42 + y`，其中包含标识符 `x`、`y`，数字 `42`，赋值号 `=`，加号 `+`，以及空格等字符。这些字符大多不在原 DFA 的字母表和状态转换表中，因此扫描器在读取这些字符时无法找到合法的状态转移，就会出现 `Invalid character` 或 `Invalid DFA state` 等错误。此外，词法分析器需要识别的是一组 Token，例如 `ID`、`NUM`、`OPERATOR` 等，而实验一的 DFA 只能判断一个完整字符串是否被接受，不能完成连续扫描、单词切分和 Token 分类。因此，第一个 DFA 不能直接作为实验二的词法规则使用。解决方法是扩展 DFA，使其能够支持标识符、数字、运算符、界符和空白符等词法单元的识别，从而正确分析源程序并输出 Token 流。